

DESAFÍO DE TRIPULACIONES · EQUIPO 3 · BBK / INETUM



SUSTRAIAPP

Agenda cultural y gastronómica del País Vasco

MEMORIA TÉCNICA

Bootcamp BBK / Inetum – Bloque Full Stack

Junio de 2026

EQUIPO DE DESARROLLO

Jefferson Aguilar

Frontend

Asier González

Backend

Olatz González

Frontend

Iker Melero

Backend

Índice

1	Introducción	3
1.1	Contexto y justificación	3
1.2	Objetivos y requerimientos	4
1.3	Impacto social y accesibilidad	5
1.4	Equipo y metodología	5
1.5	Stack tecnológico	6
1.6	Declaración sobre uso de IA generativa	6
2	Diseño y Arquitectura	7
2.1	Arquitectura general del sistema	7
2.2	Modelo de datos	8
2.3	Flujo de datos del chatbot	8
3	Implementación	9
3.1	Frontend – Chatbot asistente	9
3.2	Frontend – Panel de administración	10
3.3	Backend – Proxy del chatbot	11
3.4	DataV2 – Lógica del asistente	12
3.5	Seguridad aplicada	13

1 Introducción

SustraiApp es una aplicación web full stack que **agrega, cura y personaliza** la oferta cultural y gastronómica del País Vasco para un público de 45 a 65 años, integrando un **chatbot asistente** con base de datos en tiempo real y un **panel de administración** completo para la gestión de negocios y promociones.

1.1 Contexto y justificación del proyecto

1.1.1 Problemática y estado del arte

El País Vasco ofrece una intensa actividad cultural (exposiciones, conciertos, artes escénicas, patrimonio) y una reconocida oferta gastronómica. Sin embargo, esta riqueza está **dispersa entre múltiples fuentes** públicas y privadas, lo que obliga al usuario a consultar varias plataformas para planificar una salida.

Para el público objetivo —personas de 45 a 65 años con poder adquisitivo medio-alto— esta fragmentación se combina con barreras adicionales: interfaces poco accesibles, falta de curación según perfil y ausencia de trilingüismo.

⚠ Hueco de mercado identificado

Las **agendas institucionales** (Kulturklik, portales municipales) ofrecen información fiable pero fragmentada y sin UX moderna. Los **buscadores generalistas** tienen gran cobertura pero no curan para perfiles concretos ni integran cultura, gastronomía y patrimonio en una sola experiencia.

1.1.2 Aportación de valor

- **Agregación y curación:** unifica datos de GEO-Euskadi, Kulturklik y Google Places en un catálogo coherente, filtrable por categorías, proximidad y momento del día.
- **Personalización inteligente:** integra el modelo de recomendación y el chatbot expuestos por la API del equipo de Data Science.
- **Accesibilidad y trilingüismo por diseño:** responsive WCAG 2.1 AA con soporte completo en euskera, castellano e inglés.

- **Modelo de negocio integrado:** panel de administración con gestión de activación y promoción de negocios.

1.2 Objetivos y requerimientos

Requerimientos funcionales

Requerimiento funcional	
RF-01	Exploración de la oferta por secciones (Eventos, Gastronomía, Cultura) con secciones prefiltradas y «Ver todos».
RF-02	Filtrado por categoría y ordenación de resultados dentro de cada sección.
RF-03	Detalle de cada experiencia con su información completa y ubicación.
RF-04	Chatbot asistente como FAB flotante: detecta intención (gastro/cultura/evento) y ubicación en lenguaje natural. Devuelve ítems navegables con valoración. Funciona con y sin sesión.
RF-05	Gestión de cuenta de usuario: registro, inicio de sesión y favoritos.
RF-06	Soporte trilingüe (euskera, castellano, inglés) seleccionable por el usuario.
RF-07	Panel de administración: gestión de negocios y promociones.
RF-08	Panel mejorado: pestañas independientes (Gastro / Cultura / Eventos), búsqueda por nombre, filtro por categoría y toggles de activación y patrocinio (<code>is_sponsored</code>).

Requerimientos no funcionales

Requerimiento no funcional	
RNF-01	Accesibilidad WCAG 2.1 AA. Texto, contraste, foco visible y áreas táctiles ≥ 44 px.
RNF-02	Diseño responsive (mobile-first) en móvil, tablet y escritorio.
RNF-03	Seguridad OWASP: autorización validada en servidor en cada endpoint sensible.
RNF-04	JWT en httpOnly cookie: token sin exposición a JavaScript del cliente.
RNF-05	Mantenibilidad: convenciones en inglés, CSS BEM con tokens globales.
RNF-06	Portabilidad del entorno mediante Docker Compose .

1.3 Impacto social y accesibilidad

✓ Alineación con ODS

- **ODS 10:** accesibilidad WCAG 2.1 AA reduce barreras para personas mayores.
- **ODS 8 y 11:** visibilidad curada a la oferta cultural y gastronómica local impulsa el tejido económico.
- **ODS 4 y 10:** el euskera en igualdad con el castellano y el inglés promueve la diversidad lingüística.

1.4 Equipo y metodología

Subequipo	Integrantes	Responsabilidad principal
Backend	Asier González · Iker Melero	API REST, autenticación JWT, lógica de usuario/admin, PostgreSQL + Sequelize.
Frontend	Olatz González · Jefferson Aguilar	SPA en React + Vite, componentes, páginas, accesibilidad, internacionalización.

Se ha seguido un enfoque **ágil con entregas incrementales** coordinadas con los equipos de Data Science, Diseño y QA del Desafío de Tripulaciones.

1.5 Stack tecnológico

⚙️ React + Vite

SPA con react-router v7. Build ultrarrápido. JavaScript sin TypeScript para agilidad en el bootcamp.

🎨 CSS BEM + Tokens

CSS por componente con clases semánticas. Variables CSS globales de color, tipografía y espaciado.

📦 Node.js + Express

API REST con autenticación JWT. Middleware de roles. Sequelize ORM contra PostgreSQL.

🗄️ PostgreSQL

Modelos: usuarios, municipios, gastronomía, cultura, eventos, favoritos, cualificaciones.

🐍 Flask + Python

API de Data Science: chatbot, recomendador y endpoints de contenido. Modelos Pydantic.

🐳 Docker Compose

Orquestación de los tres servicios (frontend, backend, DataV2) en entornos reproducibles.

1.6 Declaración sobre uso de IA generativa

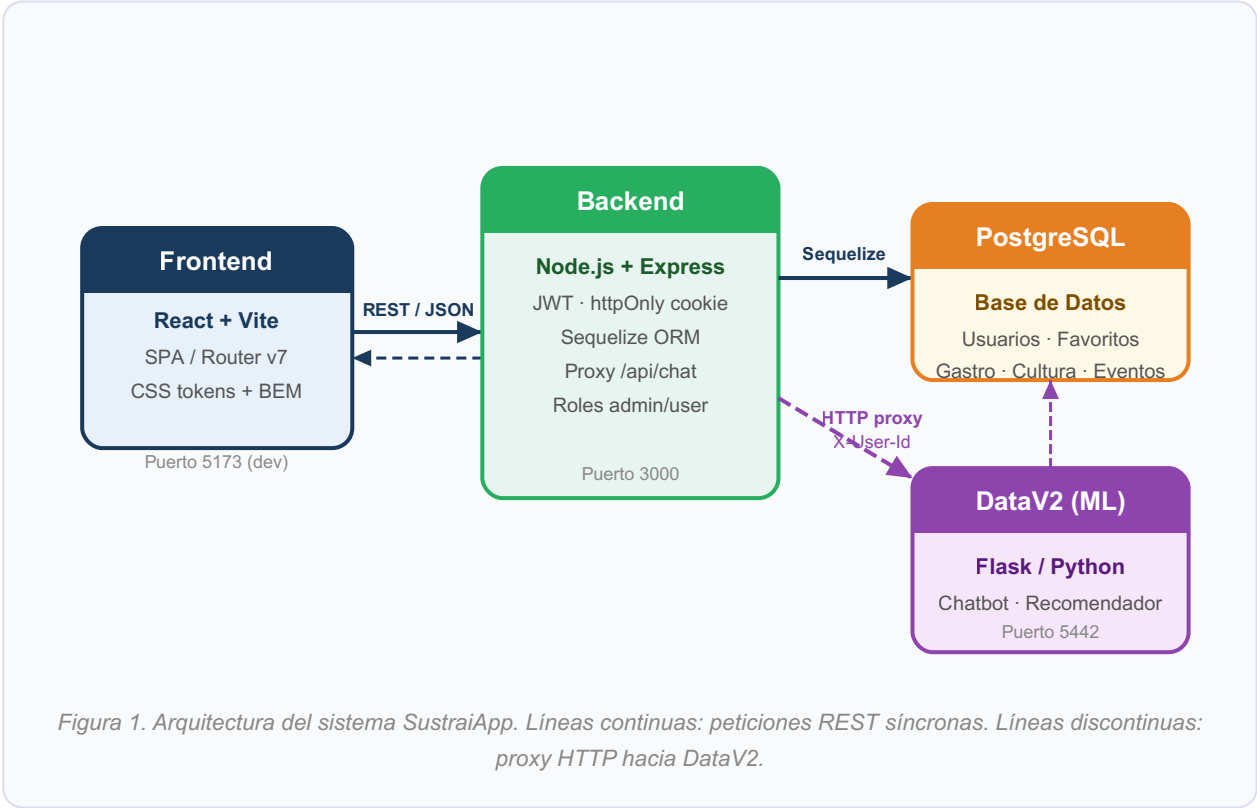
i Uso declarado de IA generativa

Se declara el uso de herramientas de IA generativa (Claude Code / Anthropic) bajo un enfoque de **asistencia instrumental**: apoyo en código repetitivo, dudas técnicas puntuales y datos sintéticos para pruebas. Todo el código ha sido revisado e integrado por el equipo, que es responsable del diseño de soluciones y las decisiones de arquitectura.

2 Diseño y Arquitectura

2.1 Arquitectura general del sistema

SustraiApp sigue una arquitectura de **cuatro capas desacopladas** que se comunican mediante APIs REST:



Capa	Tecnología	Responsabilidad
Frontend SPA	React + Vite (5173)	Interfaz, enrutado, contextos de auth/idioma/favoritos, comunicación con API.
Backend API	Node.js + Express (3000)	Auth JWT (httpOnly cookie), lógica de negocio, acceso a PostgreSQL y proxy a DataV2.
Base de datos	PostgreSQL	Persistencia de usuarios, favoritos, municipios, contenido y cualificaciones.
API ML / DataV2	Flask / Python (5442)	Chatbot conversacional, recomendaciones y listados de contenido curado.

2.2 Modelo de datos

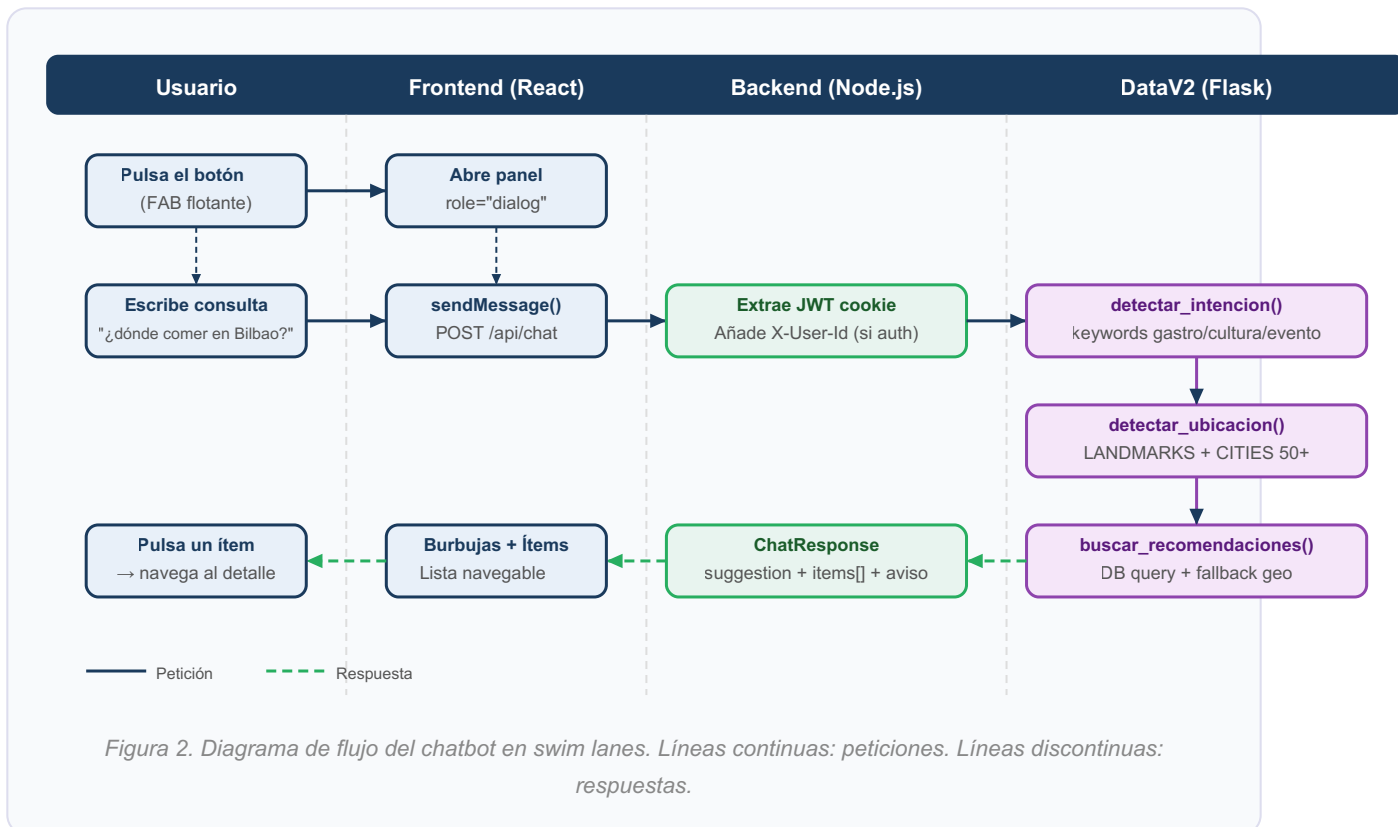
- **Usuario** — autenticación, perfil (municipio, edad, género), rol (`user` / `admin`).
- **Municipio** — lookup geográfico (nombre, provincia). Conecta con usuarios, gastro, cultura y eventos.
- **Gastronomía / Cultura / Evento** — entidades de contenido con `active` (visibilidad) e `is_sponsored` (promoción), gestionados desde el panel de admin.
- **Favorite** — relación N:M entre Usuario e ítems de contenido.
- **GastronomyQualification / Qualification** — distinciones Michelin y Repsol.
- **UserInteres / Interes** — intereses del usuario para personalización futura.

i Regla de negocio clave

Al desactivar un negocio (`active = false`), su promoción deja de mostrarse al usuario final aunque `is_sponsored` permanezca en `true`, porque todos los endpoints filtran por `active = true`.

2.3 Flujo de datos del chatbot

El chatbot recorre **tres saltos** desde que el usuario escribe su consulta hasta que recibe la respuesta con ítems navegables:



✓ Diseño resiliente: funciona con y sin sesión

El Backend extrae el `userId` del JWT de forma silenciosa: si la cookie no existe o el token ha expirado, el chatbot sigue funcionando sin personalización. El usuario no recibe un error 401.

3 Implementación

3.1 Frontend – Chatbot asistente `Chatbot.jsx`

El componente se monta en `MainLayout`, disponible en **todas las páginas** sin que la ruta afecte al estado de conversación.

Elemento	Descripción técnica
FAB flotante	<code>aria-haspopup="dialog"</code> y <code>aria-expanded</code> . Icono  en reposo,  cuando el panel está abierto.
Panel dialog	<code>role="dialog"</code> , <code>aria-modal="true"</code> . Foco transferido al input al abrir. Backdrop con click para cerrar.
Burbujas de mensaje	BEM <code>chatbot__bubble--user / --assistant</code> . Área con <code>aria-live="polite"</code> .
Ítems navegables	Cada ítem es un <code><button></code> que cierra el chat y navega a <code>/gastronomy/:id</code> , <code>/culture/:id</code> o <code>/events/:id</code> según <code>subtipo</code> .
Indicador de escritura	Tres puntos animados (<code>chatbot__typing</code>) mientras se espera respuesta.
<code>session_id</code>	Generado al cargar la app (<code>session-<timestamp></code>) para contexto conversacional futuro en DataV2.

i Accesibilidad WCAG 2.1 AA

Todos los controles tienen `aria-label`. El área de mensajes usa `aria-live="polite"`. Áreas táctiles $\geq 44 \times 44$ px. Ratio de contraste de texto $\geq 4.5:1$.

3.2 Frontend – Panel de administración `AdminComerciosPage.jsx`

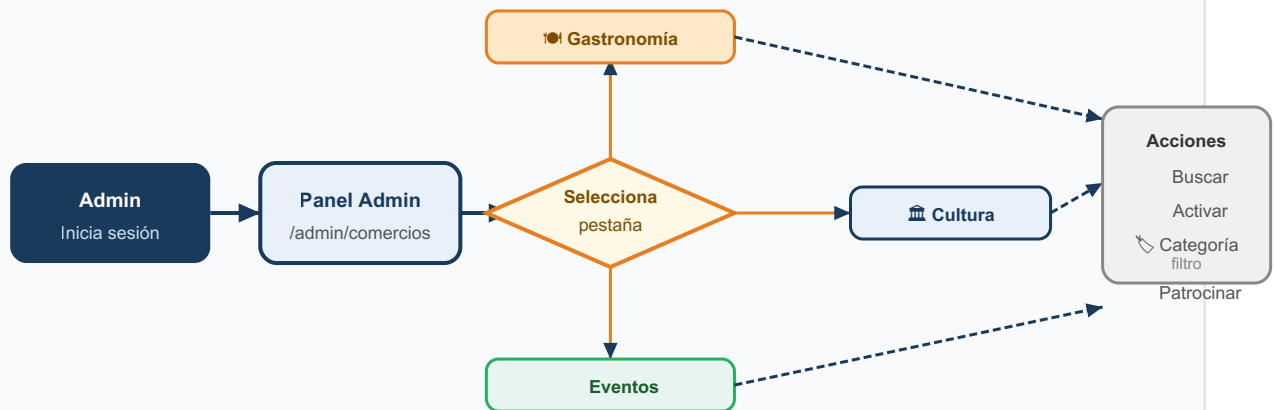


Figura 3. Flujo del administrador en el panel de gestión de contenido.

Característica	Implementación
Pestañas con lazy load	Set <code>loadedTabs</code> . Primera activación lanza la petición; visitas posteriores reutilizan el estado sin nueva petición.
Búsqueda + filtro reactivos	<code>useMemo</code> sobre el array activo: nombre con <code>toLowerCase()</code> y categoría por pestaña (<code>tipo_comida</code> , <code>tipo_lugar</code> , <code>type</code>). Sin peticiones al servidor.
Toggle Activar/Desactivar	Actualización optimista con <code>{ active: !item.active }</code> . Estado de carga por ítem (<code>updating = id + field</code>).
Toggle Patrocinar	Mismo mecanismo con <code>{ is_sponsored: !item.is_sponsored }</code> . Badge visual cuando está activo.
Protección de ruta	Redirección a <code>/</code> si <code>user.role !== "admin"</code> en el primer render post-auth.

3.3 Backend – Proxy del chatbot `chat.routes.js`

- **Validación:** rechaza con `400` si el cuerpo no contiene `message`.
- **Extracción silenciosa de identidad:** verifica el JWT de la cookie `access_token`. Si es válido adjunta `X-User-Id`; si falla, continúa sin personalización.
- **Proxy con timeout:** reenvía a `ML_API_URL/api/chat` con timeout de 15 000 ms. Propaga el código HTTP de DataV2.

✓ Opacidad del endpoint externo

El cliente nunca conoce la URL ni el puerto de DataV2. Toda la comunicación pasa obligatoriamente por el backend Node.js.

3.4 DataV2 – Lógica del asistente `assistant_logic.py`

Detección de intención y ubicación

Intención	Palabras clave representativas	Ejemplo de consulta
Gastro	restaurante, pintxos, comer, sidrería, michelin, menú, txikiteo...	«¿Dónde comer en Bilbao?»
Cultura	museo, patrimonio, castillo, visita, ermita, casco histórico...	«Qué ver en Vitoria»
Evento	concierto, festival, agenda, hoy, fin de semana, en euskera...	«Qué hay este finde»
General	plan, ocio, qué hacer, recomendación, turista, escapada...	«Ideas para el finde»

★ LANDMARKS (específico) → CITIES (general)

Primero se buscan **landmarks** (Guggenheim, La Concha, San Mamés, Artium...); si no hay coincidencia, se busca en **CITIES**: más de 50 municipios en euskera, castellano y apodos: *Donosti / Donostia / San Sebastián / Sanse, Bilbo / La Villa, Gasteiz / Vitoria-Gasteiz*.

Consulta con fallback geográfico

1. Filtra por **municipio exacto** (`ILIKE '%municipio%'`).
2. Si vacío, amplía a **toda la provincia**.
3. Si vacío, devuelve los mejor valorados **sin filtro geográfico**.

Respuesta estructurada (Pydantic)

Campo	Tipo	Descripción
<code>suggestion</code>	string	«Cerca de Bilbao te recomiendo estas opciones de gastronomía...»
<code>items[]</code>	List[Item]	Ítems con <code>item_id</code> , <code>nombre</code> , <code>subtipo</code> , <code>categoria</code> , <code>provincia</code> , <code>estrella_prevista</code> .
<code>aviso</code>	string?	Aviso opcional (resultados limitados, fuera de zona...).

3.5 Seguridad aplicada

i Medidas de seguridad transversales (OWASP)

- **JWT en httpOnly cookie:** token nunca accesible desde JavaScript del cliente (XSS).
- **Proxy opaco del chatbot:** URL y puerto de DataV2 no expuestos al cliente.
- **Silent JWT failure:** token inválido no genera 401 visible; simplemente omite la personalización.
- **Doble validación de rol admin:** redirección en cliente (UX) + middleware de servidor en todas las rutas `/admin/*`.
- **Toggles validados:** los cambios de `active` e `is_sponsored` pasan por el middleware antes de persistirse.
- **Timeout en proxy:** límite de 15 s previene que una respuesta lenta de DataV2 bloquee Node.js.